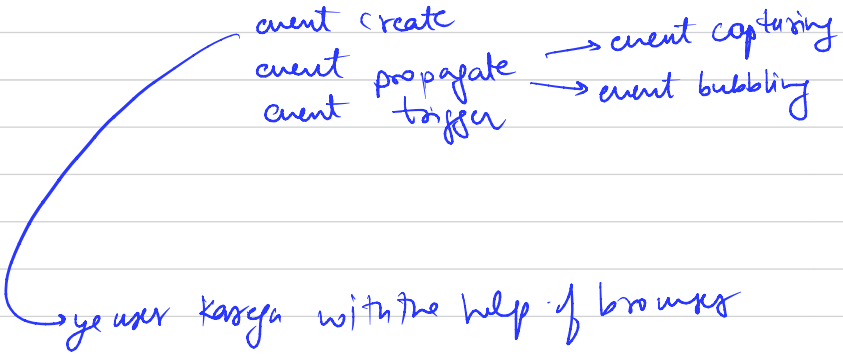
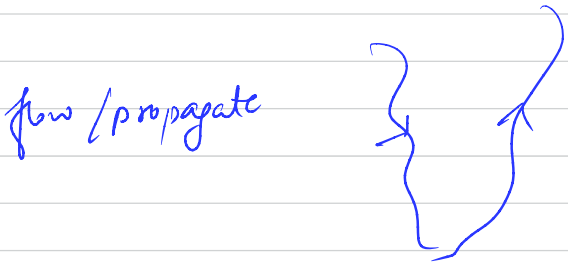
Event life cycle :-





- capture value → By default ye false hota hai.
  - ↳ by default false hota hai.

e.stop → to stop event from propagating
- ↳ e.stopPropagation()

Event Delegation :-

Dynamically add karne par event trigger nahi hoga.

parent par lagade → instead of individual div.

Homework

Don't use class or toggle to select or deselect.

Personal Notes

6 Aug, 2024

Reference :- Shreyance 17

Event life cycle :-

video starts from 16:00

{ file name: Class 21 Homework
Event life cycle }

Html code

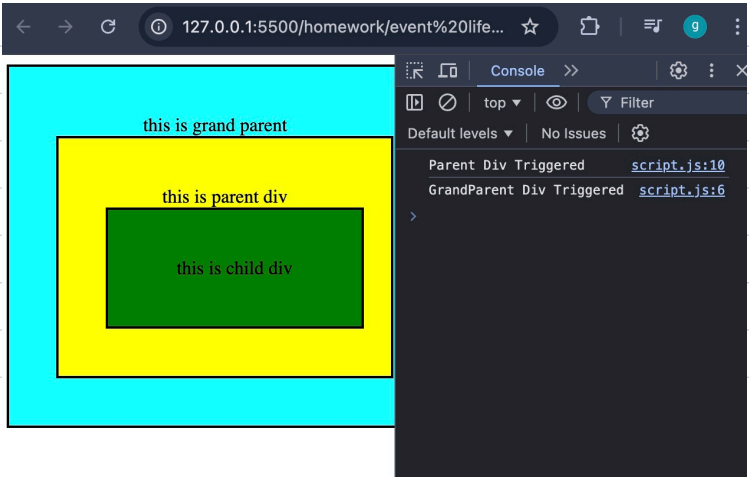
```
<div class="grand-parent">
        this is grand parent
        <div class="parent">
            this is parent div
            <div class="child">
                this is child div
            </div>
        </div>
    </div>
```

JS code

```
const grandParent = document.querySelector('.grand-parent');
const parent = document.querySelector('.parent');
const child = document.querySelector('.child');

grandParent.addEventListener('click', (e)=>{
    console.log('GrandParent Div Triggered');
})

parent.addEventListener('click', (e)=>{
    console.log('Parent Div Triggered');
})

child.addEventListener('click', (e)=>{
    console.log('Child Div Triggered');
})
```

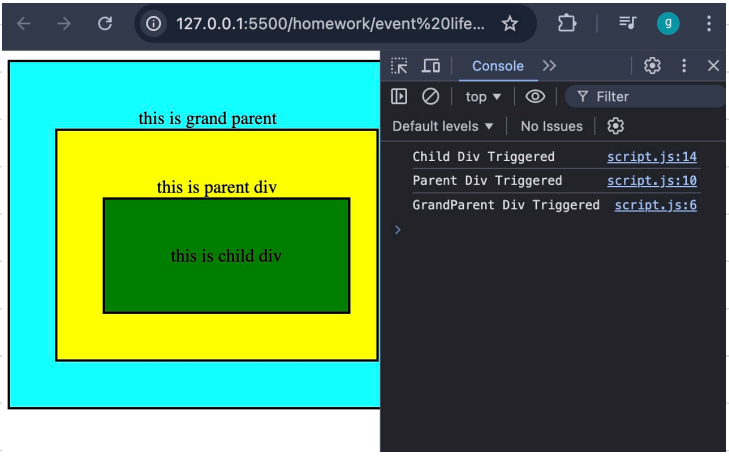
→ Agar mai grand parent div par click karta hu, then o/p hoga "grand parent div triggered"

→ Agar mai "parent div" par click karunga, then o/p hoga → "parent div"
"grand parent div"



→ Agar mai click karunga on "childdiv"

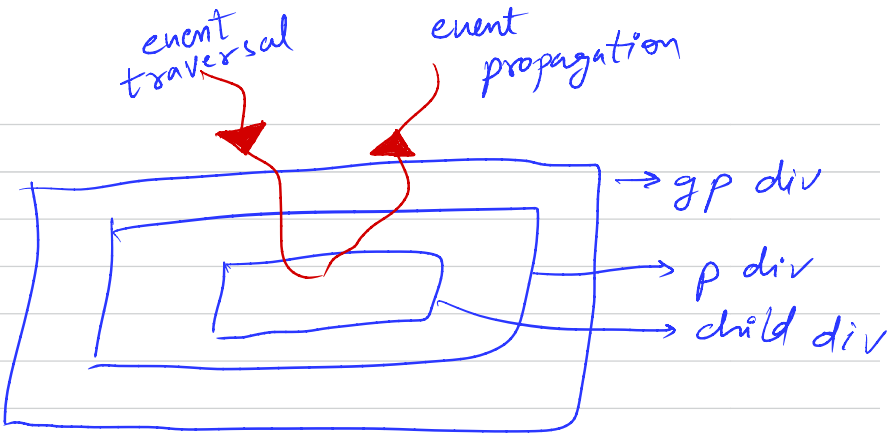
o/p hoga → child div
parent div
grand child div



But aisa ho kyu raha hai?

Ise event bubbling kehte hai. jab mai childdiv par click kar raha hu, i am also clicking on common area b/w childdiv, parent & grand parent div.

Kuch area common hone ke karan aisa ho raha hai.



event traversal → going inside

event propagation → coming outside

Let's say maine childdiv par click kiya, tab ye pehle use dhundega and event trigger karega, then bahar aate time baaki ke events ko trigger karta hua jayega.

By default event capturing value false hoti hai.

```
const grandParent = document.querySelector('.grand-parent');
const parent = document.querySelector('.parent');
const child = document.querySelector('.child');

grandParent.addEventListener('click', (e)=>{
    console.log('GrandParent Div Triggered');
},false)

parent.addEventListener('click', (e)=>{
    console.log('Parent Div Triggered');
},false)

child.addEventListener('click', (e)=>{
    console.log('Child Div Triggered');
},false)
```

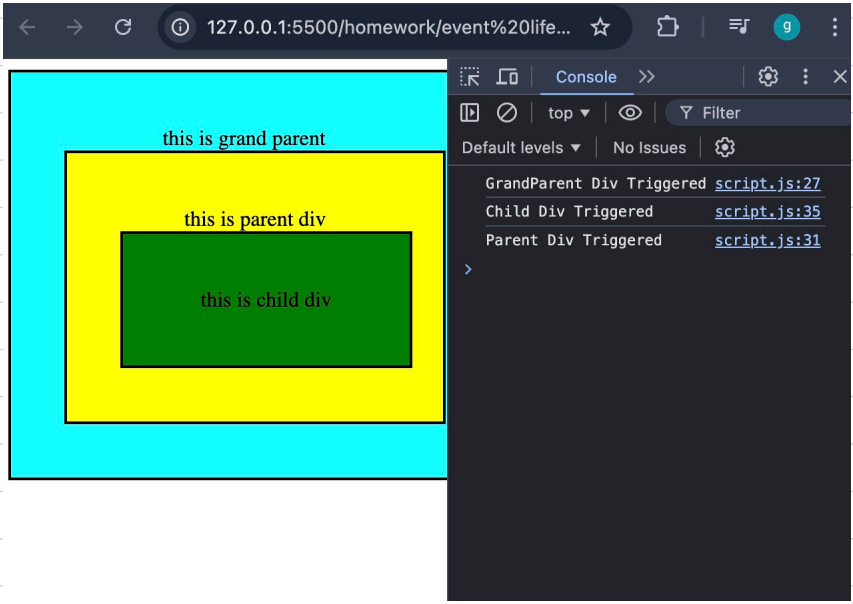
Q Mai chahta hu event traversal ke time event trigger ho, uski event propagation ke time. ye kaise hoga?

Ans Jisko event traversal ke during trigger karana hai, usha event capturing value true kar de.

gp → true

parent → false

child → true

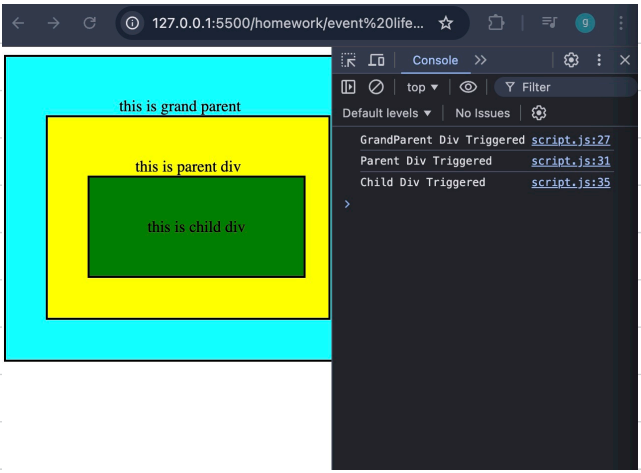


Q

```
const grandParent = document.querySelector('.grand-parent');
const parent = document.querySelector('.parent');
const child = document.querySelector('.child');

grandParent.addEventListener('click', (e)=>{
    console.log('GrandParent Div Triggered');
},true)

parent.addEventListener('click', (e)=>{
    console.log('Parent Div Triggered');
},true)

child.addEventListener('click', (e)=>{
    console.log('Child Div Triggered');
},true)
```

Q Agar teeno true karde tab kya hoga?

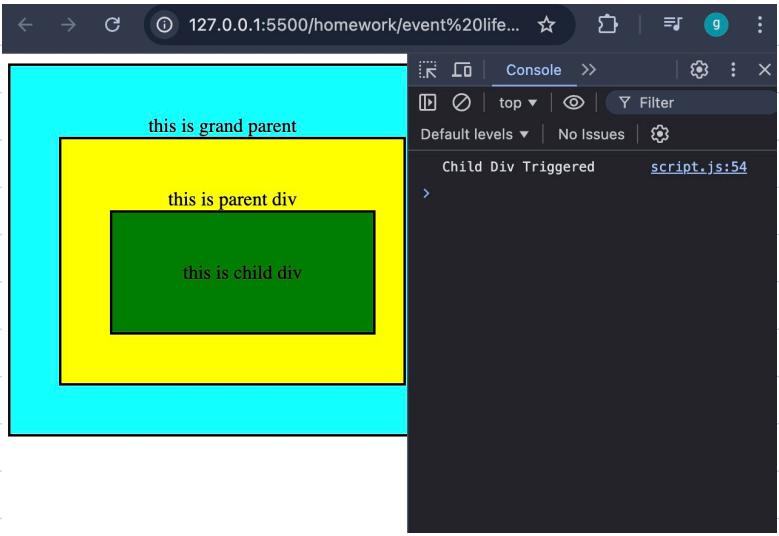
↳ o/p hoga → GP Div
Parent Div
Child Div



e.stopPropagation() → to stop event from propagating

Q

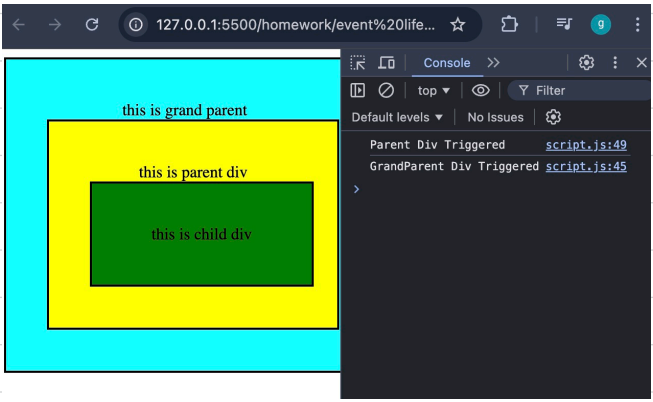
```
const grandParent = document.querySelector('.grand-parent');
const parent = document.querySelector('.parent');
const child = document.querySelector('.child');

grandParent.addEventListener('click', (e)=>{
    console.log('GrandParent Div Triggered');
},false)

parent.addEventListener('click', (e)=>{
    console.log('Parent Div Triggered');
},false)

child.addEventListener('click', (e)=>{
    e.stopPropagation();
    console.log('Child Div Triggered');
},false)
```



Child par e.stopPropagation() laga hai, iska matlab child ke baad propagation nahi hoga, isliye sirf "child div" o/p mei aaya.

Q for same code (pasted above) agar mai parent div par click karunga, then "parent and grandparent div" aayega

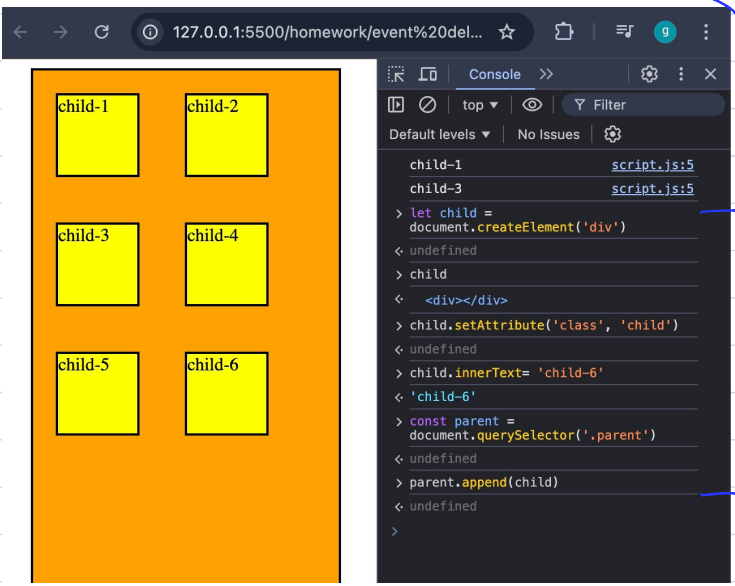
o/p


127.0.0.1:5500/homework/event%20life...
this is grand parent
this is parent div
this is child div
Console
top
Filter
Default levels
No Issues
Parent Div Triggered
script.js:49
GrandParent Div Triggered
script.js:45

Event Delegation :-

Let's say maine 5 div banaye and unn par click karne se div ka text mujhe console mei mil raha hai.

Maan lo mujhe 6th div dynamically add karna hai. We did that.



But child-6 div par click karne par child-6 console mei ~~xxxx~~ nahi aaraha. How to solve this problem?

parent par bubbling laga do. Ab jab child-6 click hoga, it means parent container / div bhi click hoga.

```
const childs = document.querySelectorAll('.child');

for(let child of childs){
    child.addEventListener('click', (e)=>{
        console.log(e.target.innerText);
    })
}
```

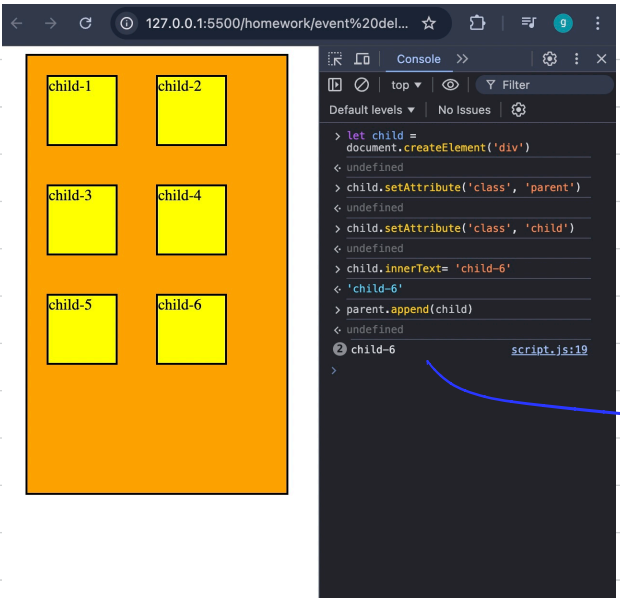
→ we will avoid using it.

Pehle hum ye samajhte hai ki child 6 par click karne par "child-6" console mei kyu nahi aaya.

Pehle html code chalega then uske bottom mei likhi huyi script file chalegi, then browser mei add kiya code chalega. Hamari script file already run ho chuki jis karan jis karan child-6 par click karne se kuch nahi hoga.

```
const childs = document.querySelectorAll('.child');
const parent = document.querySelector('.parent');

parent.addEventListener('click', (e)=>{
    console.log(e.target.innerText);
})
```

→ Hamne parent container par event add kar diya. Ab agar dynamically div add karke uspe click karoge. O/p aayega



child-6 is there. It was dynamically added.

```
const parent = document.querySelector('.parent');
const childs = document.querySelectorAll('.child');
console.log('hi')

parent.addEventListener('click', (e)=>{
    if(e.target.classList.contains('child')){
        console.log(e.target.innerText);
    }
})
```

↳ this is better than the code pasted on last page

The reason we avoided loop method is bcoz in loop method, we were selecting all the div elements inside main div. Also dynamically create kiya

gya element par event lag nahi saka tha.

Reason → why this code is giving o/p for child-6

```
const childs = document.querySelectorAll('.child');
const parent = document.querySelector('.parent');

parent.addEventListener('click', (e)=>{
    console.log(e.target.innerText);
})
```

Here we have not added any event for child-6, still it is working for child-6. why?

Here's a step-by-step explanation:

Event Delegation: You have added an event listener to the parent element. This means that any click event on the parent or its children will bubble up to the parent, and the parent will handle it.

Event Bubbling: When you click on a child element, the event first triggers on the child element and then bubbles up to its parent elements. So, even though the click happens on the new child element (child-6), the event will bubble up to the parent element, which has the event listener.

Dynamic Elements: Event delegation works well with dynamically created elements. Even if you add new child elements after setting up the event listener on the parent, the event listener will still handle events on these new elements because it listens for events on the parent, not on individual children.

Event Target: The e.target property in the event handler refers to the element that was actually clicked. So, when you click on the dynamically created child-6 element, e.target will be this new element.

Note:- youtube se event life cycle & event delegation sahi se samjho.

Important:-

ye topic (Event Bubbling, Event Capturing) study from GFG

https://www.geeksforgeeks.org/what-is-event-bubbling-and-capturing-in-javascript/?ref=ml_lbp

→ Reference link

Let's try to understand it with an example:-

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content=
        "width=device-width, initial-scale=1.0">
    <title>
        GeeksForGeeks: What is Event
        Bubbling and Capturing?
    </title>
</head>

<body>
    <div class="button-list"
        onclick="console.log('button-list')">
        <div class="button-container"
            onclick="console.log('container')">
            <button class="button"
                onclick="console.log('button')">
                Click me!
            </button>
        </div>
    </div>
</body>

</html>
```

→ this is the code

when you click on button with "click me!"

you get → button
container
button-list

Let's understand how it is working :-

① Whenever you interact with a DOM element, the browser creates a new DOM Event. In this case, when clicking on the button, the browser composes a specific DOM Event: the PointerEvent.

② Dispatching an event :-

After creating the Event object, the browser dispatches it. The event then traverses the DOM in what is called Event Propagation. There are three phases in the Event Propagation process.

Capture phase: The event starts from the root of the DOM, and starts traversing down the tree until it finds the element that triggered the event. For each element it traverses, it checks if there's an Event Listener with the capture flag set to true, attached to it. If not, it just ignores it. Note that in our example, the first log message we saw in the console was the button message. That's because, by default, event listeners do not listen for events on the capture phase.

Target phase: The event reaches the target element, and runs the event listener attached to it, if any. This phase got us the first message in the console in our example: "button"

Bubbling phase: Similar to the capturing phase, the event will traverse the DOM checking for Event Listeners, but this time it will start in the element that dispatched the event and finish on the DOM's root (the html element). It is in this phase that we got the logs "container" and "button list" in our example, because they are the closest ancestors to the button in that order! Note that Event Listeners, by default, will be called in this phase.

Please check the image on the next page.

Event propagation of the "click" event through the example DOM

Event Delegation :-

Event delegation is a technique in JavaScript that allows you to manage events efficiently. Here's a simple explanation:

Basic Concept

Normally, when you want to handle an event, you attach an event listener to each element that you want to respond to that event. For example, if you have a list of items (<li> elements) and you want to detect clicks on each item, you might add a click event listener to each <li> element.

Problem

Adding an event listener to every single element can be inefficient, especially if you have a large number of elements. It can also be cumbersome to manage.

Event Delegation Solution

Instead of adding event listeners to multiple elements, you add a single event listener to a parent element of those items. Because events in JavaScript bubble up from the target element (the one that was actually clicked) to the root of the document, you can listen for the event on a parent element and determine which child element triggered the event.

How It Works

Attach an Event Listener to the Parent Element:

Instead of attaching an event listener to each child element, you attach it to a parent element.

Use Event Bubbling:

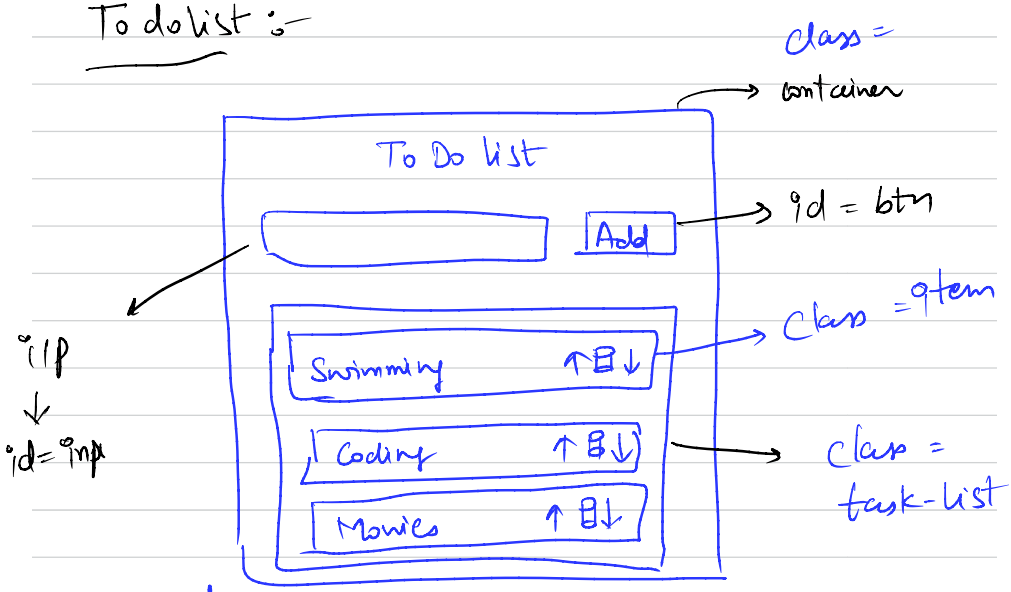
When an event occurs on a child element, the event bubbles up to the parent element. The parent element's event listener can then handle the event.

Identify the Target:

Inside the event handler, you can identify which child element was the actual target of the event using event.target.

Shreyance - 16 (02:08:19) 7 Aug, 2024

To do list :-

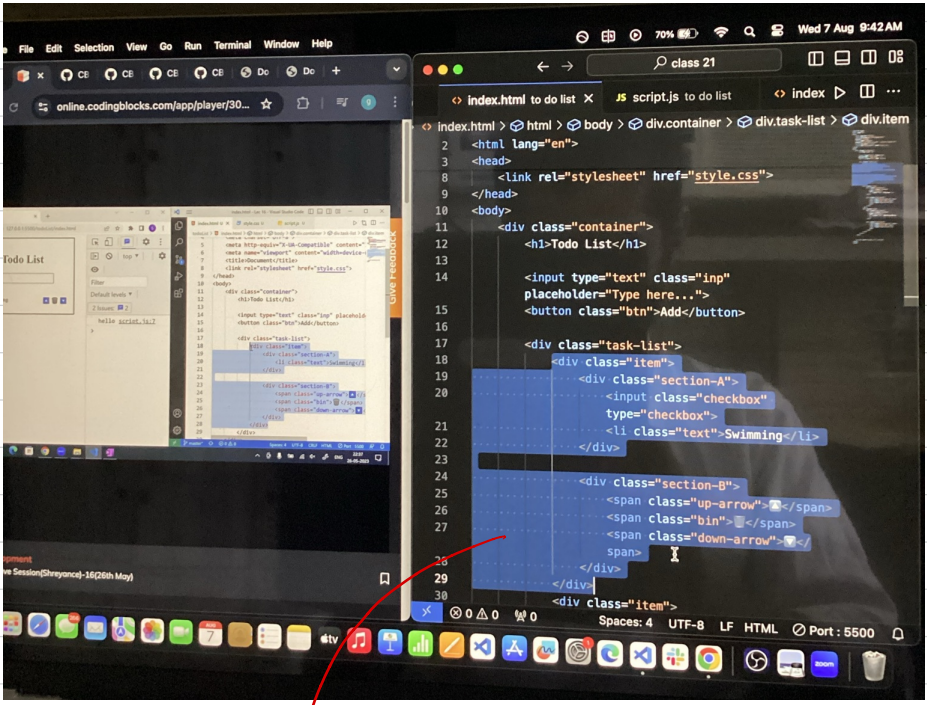


Approach :-

Sabse pehle pen and paper use karke structure banao. Structure banane ke baad class and id dedo elements ko. Then html ka code likho, then css, then js.

↳ ye part hum dynamically create karenge inside JS but for bhi aapko yo part html code mei likhna chahiye but comment

kaske.



ye code comment hoga, then
ise hum dynamically
create karenge.

```
const inp = document.querySelector('.inp');
const btn = document.querySelector('.btn');
const taskList = document.querySelector('.task-list');

// when clicking on add button

btn.addEventListener('click', (e)=>{
    console.log(inp.value);
    const newItem = document.createElement('div');
    newItem.setAttribute('class', 'item');

    let str = `<div class="section-A">
                    <input class="checkbox" type="checkbox">
                    <li class = "text"> ${inp.value}</li>
                </div>

                <div class="section-B">
                        <span class="up-arrow">🔼</span>
                        <span class="bin">🗑️</span>
                        <span class="down-arrow">🔽</span>
                </div>`


    newItem.innerHTML = str;
    inp.value = '';
    taskList.append(newItem);

});


// when clicking on task list

taskList.addEventListener('click', (e)=>{
    // console.log(e.target);

    const element = e.target.getAttribute('class');
    // console.log(element);

    if(element === 'bin'){
        const item = e.target.parentElement.parentElement
        // console.log(item);
        item.remove();
    }

    else if(element === 'up-arrow'){
        const currItem = e.target.parentElement.parentElement
        const prevItem = currItem.previousElementSibling;

        console.log(prevItem);
        prevItem.before(currItem);
    }

    else if(element === 'down-arrow'){
        const currItem = e.target.parentElement.parentElement
        const nextItem = currItem.nextElementSibling;

        console.log(nextItem);
        nextItem.after(currItem);
    }

    else if(element === "checkbox"){
        console.log(e.target);
        const textItem = e.target. nextElementSibling;
        console.log(textItem);
        textItem.classList.toggle('checked');
    }
});
```

Explanation :-

① Jaise hi add button par click ho. Vaise hi dynamically list item add ho jaye inside task-list. New list item is being created dynamically

l1, l2 par ek div create kiya and usko class de di → "item"

Whatever value you put in input, it will go to new item with html code.

Then inp. value empty ho gyi.

new item ko append kar diya inside tasklist.

② task list mei jaha bhi click kiya ja raha hai us element ki class store kar lo (l3)

③ Maan lo maine item delete karna chaha then "bin" class ka parent "section-B" hai and "section B" ka jo parent hai → wo delete ho gaa which is "item".

l4 par sirf "item" class wala div delete ho raha hai

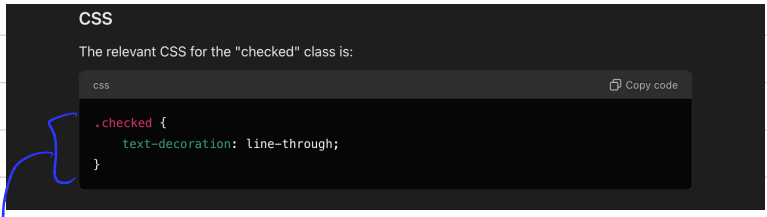
④ l5, l6 par element ko upar neeche karne ka bataya.

(5) l7 jaise hi check box check hoga, text ko line se cut kar do.

'toggle ('checked')' → it adds the 'checked' class to the text item if it is not present and removes it if it is present.

what is exactly l8 is doing? ↓

This line effectively toggles the "checked" class on the <li> element containing the task text. When the class is added, the text will have a line-through style (as defined in your CSS with text-decoration: line-through;), indicating that the task is completed. When the class is removed, the text returns to its normal style.



this is added in CSS code